

pre_build_verification.sh

pre_build_verification.sh — companion guide

Revision: 4 Last modified: 2026-06-14T18:45:00Z

Overview

scripts/pre_build_verification.sh is the project's pre-build gate-suite runner (WORKING_PLAN item P4.2). It runs a small set of GENUINE, mechanically-checkable CM-* gates BEFORE every build, per the inherited Constitution's **MANDATORY TESTING CONSTRAINTS** ("Pre-build / pre-merge verification — BEFORE every build"). It is the first real CM-* gate runner for this project; before P4.2 only the constitution-inheritance meta-test existed.

Each gate fails ONLY on a real, user-visible repository-hygiene violation — never on a script-internal bug (§11.4.1) — and each is backed by a PAIRED §1.1 mutation test under scripts/testing/ that proves the gate is not a bluff.

Implemented gates

Gate ID	Anchor	What it asserts
CM-GITIGNORE-PRECOMMIT-AUDIT	§11.4.30	No tracked file matches the forbidden build-artifact / secret classes the project's .gitignore declares. Includes the cmd/api-server anchoring regression guard : the prebuilt root api-server binary must stay anchored as /api-server so the bare token never re-hides the SOURCE directory.
CM-NO-FAKES-BEYOND-UNIT	§11.4.27	Non-unit Go test files (build-tagged integration/e2e/stress/performance/security) do not import a mock/stub/fake package path. Mocks are permitted in unit tests only.
CM-SCRIPT-TARGET-SHELL-PARSEABLE	§11.4.67	Every tracked *.sh under scripts/ + scripts/testing/ parses cleanly under bash -n; any script declaring an sh-family shebang (#!/bin/sh, #!/usr/bin/env sh) or no recognised shell shebang must ALSO parse under sh -n. A script with an honest bash shebang is only required to be bash-parseable (invoked

Gate ID	Anchor	What it asserts
CM- VERSION- SINGLE- SOURCE	P0.1 / a36030e	via its shebang it never runs under sh), so its bash-only constructs (mapfile, < <(...), [[]], arrays) are legitimate and must NOT be false-FAILED. No cmd/*/main.go declares a hardcoded semver version literal (e.g. appVersion = "3.0.0"); every binary's version derives from pkg/version.AppVersion (== authoritative VERSION file). A fast grep complement of the Go test TestNoBinaryDeclaresDivergentVersionLiteral.
CM- TRACKER- DOCS- PRESENT	§11.4.15 / .16 / .53 / .44	The four canonical workable-item tracker docs (docs/Issues.md, docs/Fixed.md, docs/Issues_Summary.md, docs/Fixed_Summary.md) all exist AND each carries a §11.4.44 revision header — both a **Revision:** line and a **Last modified:** line in its header block. A missing doc or a missing header line is a documentation-layer violation (the tracker constellation is incomplete / unversioned).
CM-ATM- TICKET-IDS	§11.4.54	Every ###/## §X. workable-item heading in docs/Issues.md + docs/Fixed.md carries an [ATM-NNN] token; the union of all ids is unique and monotonic with no gaps — the contiguous sequence ATM-001. .ATM-NNN. A heading without a token, a duplicate id, or a gap in the sequence is a §11.4.54 violation.
CM-DOC- SIBLING- SYNC	§11.4.65	Every in-scope tracked *.md (project-root *.md, docs/**, scripts/** companions; EXCLUDING owned-submodule trees + build/out/dist/external/prebuilts/node_modules/vendor/qa-results) has BOTH a tracked .html AND a tracked .pdf sibling, and each sibling's mtime is >= the .md mtime. A missing or stale (older) sibling is a §11.4.65 universal-Markdown-export violation. The exclusion set mirrors scripts/testing/sync_all_markdown_exports.sh (the generator) so gate and generator agree on scope.
CM-NO- FORCE- PUSH- ABSOLUTE	§11.4.113	No tracked script under scripts/ contains an actual force-push invocation: a git push carrying --force / --force-with-lease / -f, OR a git push with a leading-+ forced refspec (e.g. git push origin +main:main). Force-push is STRICTLY FORBIDDEN with no exception. Comment lines, case-pattern arms, and die/echo refusal strings (the commit_all.sh §11.4.113 GUARD) are NOT invocations and do NOT trip the gate (anti-FAIL-bluff, §11.4.1).

Prerequisites

- sh (or bash) and git on PATH. grep, head.
- Run from anywhere inside the repo; the script resolves the git toplevel.
- No pandoc / weasyprint / go required — the suite is git-index + grep only, and runs in well under a second (no build, no network).

Usage examples

```
# Run the whole suite (CI / pre-build hook):  
scripts/pre_build_verification.sh
```

```
# List gate ids:  
scripts/pre_build_verification.sh --list
```

```
# Run a single gate:  
scripts/pre_build_verification.sh --gate CM-GITIGNORE-PRECOMMIT-  
AUDIT
```

Exit codes: 0 all selected gates passed · 1 at least one real violation · 2 usage / harness error (unknown gate, not a git repo).

Edge cases & honest limitations (§11.4.6)

- **CM-GITIGNORE-PRECOMMIT-AUDIT** mirrors the classes this project's .gitignore declares (the named build/* outputs, real .env files, config_with_keys.json / api_keys.json / secrets.* / keys.*). It does NOT flag every conceivable build artifact (e.g. arbitrary *.exe), because the project's declared policy is class-specific and the working tree is compliant with that declared policy. Widening the forbidden set is future work and would require the tree to be brought into compliance first.
- .env.example is explicitly allow-listed (the project's .gitignore carries !.env.example) and must NOT be flagged.
- **CM-NO-FAKES-BEYOND-UNIT** is line-grep based, not AST-based. It matches import-path string literals ending in /mock(s), /stub(s), /fake(s) in files carrying a non-unit build tag in their first 15 lines. A mock referenced indirectly (a re-exported type from a differently-named package) is NOT caught. Untagged _test.go files are treated as unit tests (mocks allowed), matching the convention that integration/e2e suites carry build tags. This is a high-value cheap probe, not a complete proof; AST-grade enforcement is tracked as future work.
- **CM-SCRIPT-TARGET-SHELL-PARSEABLE** is faithful to §11.4.67's "every shell script that may be invoked under a target shell OTHER than the one in its shebang MUST parse cleanly under that target shell." It does NOT force sh -n on an honest-bash script, because such a script invoked via its shebang only ever runs under bash — doing so would itself be a §11.4.1 FAIL-bluff (the project's scripts/demo-all.sh is exactly this case: genuine bash with mapfile + process substitution, correctly PASSes). It requires bash AND sh on PATH; if either is absent it returns 2 (harness error), never a false PASS. Scope is the tracked scripts/ tree (incl. scripts/testing/).

- **CM-VERSION-SINGLE-SOURCE** is grep-based: it matches a Go assignment of a 3-part semver string literal to an identifier named `(app)version`. A version built at runtime (`fmt.Sprintf`) or held in a differently-named identifier is NOT caught; a compliant `appVersion = version.AppVersion` reference and a 2-part XML/EPUB attr (`version="1.0"`) are correctly NOT flagged. It and the Go test `TestNoBinaryDeclaresDivergentVersionLiteral` together are the high-value probe, not an AST-grade proof.
- **CM-TRACKER-DOCS-PRESENT** checks the §11.4.44 header over the first 12 lines of each doc (where the header block lives). It asserts presence of the two header lines and the four files; it does NOT validate the revision *value* or the timestamp *freshness* (that is §11.4.44 / §11.4.60 sync-gate territory, owned by the markdown-export sync path). A well-formed constellation is never false-FAILED.
- **CM-ATM-TICKET-IDS** matches workable-item headings as lines of the form `^#{2,3} §` (the project's `§X.` heading convention). Non-item headings (the doc H1, --- rules) are not items and are not required to carry an id, so they are never false-FAILED (§11.4.1). It is a fast grep probe faithful to the tree's actual heading convention (Issues §1..N, Fixed §1..N); the unique + contiguous-001..N invariant is computed from the union of both files.
- **CM-DOC-SIBLING-SYNC** operates over TRACKED files only (`git ls-files`): an untracked stray `.md` is not a versioned doc, and an untracked sibling does not satisfy the export mandate (the export must be committed). The presence arm (`.html + .pdf` both tracked) is the durable cross-checkout invariant; the mtime arm (`sibling -nt-not-older`) is a working-tree freshness guard — honest boundary (§11.4.6): a fresh clone checks out arbitrary mtimes, so the mtime arm asserts "in THIS working tree the sibling is not older than its `.md`," exactly what the §11.4.65 generator enforces on every sync. Owned submodule trees (they own their own exports) and `build/vendor/qa` dirs are excluded, mirroring `sync_all_markdown_exports.sh`, so an excluded `.md` without siblings is never false-FAILED.
- **CM-NO-FORCE-PUSH-ABSOLUTE** scans only lines that look like a `git push` COMMAND carrying a force token, and explicitly excludes the three GUARD forms the project's own `commit_all.sh` §11.4.113 refusal logic uses: comment lines (`#-led`), case-pattern arms (a token-list ending in `)` with no `git push` verb), and `die/echo` refusal strings (or any line referencing 11.4.113). This anti-false-positive design is load-bearing — naming a forbidden flag in a GUARD is NOT an invocation, and false-FAILing it would itself be a §11.4.1 FAIL-bluff. The `+refspec` form is matched as a whitespace-led `+<word>` token so `git push origin +main:main` (where `+` follows the remote, not push) is caught.

Internal behaviour

The script is read-only over the `git` index + working tree (no writes, no build, no network). It is written in POSIX-portable shell (no arrays, no `[ [ ]`), no process substitution) and parses cleanly under both `bash -n` and `sh -n` per §11.4.67.

The mutation tests build disposable `git` repos under `mktemp` and point the gate at them via the `PBV_REPO_ROOT` env var, so the real working tree is never mutated.

Related scripts

- `scripts/testing/meta_test_gitignore_precommit_audit.sh` — paired §1.1 mutation proof for CM-GITIGNORE-PRECOMMIT-AUDIT.
- `scripts/testing/meta_test_no_fakes_beyond_unit.sh` — paired §1.1 mutation proof for CM-NO-FAKES-BEYOND-UNIT.
- `scripts/testing/meta_test_script_target_shell_parseable.sh` — paired §1.1 mutation proof for CM-SCRIPT-TARGET-SHELL-PARSEABLE.
- `scripts/testing/meta_test_version_single_source.sh` — paired §1.1 mutation proof for CM-VERSION-SINGLE-SOURCE.
- `scripts/testing/meta_test_tracker_docs_present.sh` — paired §1.1 mutation proof for CM-TRACKER-DOCS-PRESENT.
- `scripts/testing/meta_test_atm_ticket_ids.sh` — paired §1.1 mutation proof for CM-ATM-TICKET-IDS.
- `scripts/testing/meta_test_doc_sibling_sync.sh` — paired §1.1 mutation proof for CM-DOC-SIBLING-SYNC.
- `scripts/testing/meta_test_no_force_push_absolute.sh` — paired §1.1 mutation proof for CM-NO-FORCE-PUSH-ABSOLUTE.
- `scripts/testing/sync_all_markdown_exports.sh` — the §11.4.65 export generator whose exclusion set CM-DOC-SIBLING-SYNC mirrors.
- `scripts/testing/meta_test_constitution_inheritance.sh` — the pre-existing inheritance meta-test (sibling discipline).
- `scripts/commit_all.sh` — the authorised commit + push wrapper.

Last verified: 2026-06-14 (all 8 gates PASS on current tree; all eight paired mutation tests PASS — every gate FAILs on a real violation and PASSes when restored. CM-DOC-SIBLING-SYNC catches a missing `.html`, a missing `.pdf`, and a stale (backdated) sibling without false-FAILing excluded owned-submodule docs; CM-NO-FORCE-PUSH-ABSOLUTE catches `--force`, `--force-with-lease`, and a `+refspec` push without false-FAILing the `commit_all.sh`-class force-push GUARD; CM-TRACKER-DOCS-PRESENT catches a missing doc + a missing Revision/Last-modified header; CM-ATM-TICKET-IDS catches missing tokens, duplicate ids, and sequence gaps without false-FAILing a non-item heading; and the prior negatives confirm honest-bash scripts and the compliant `version.AppVersion` form are never wrongly flagged).